

Synthetic Warehouse Dataset — Setup Guide

Generate a labelled warehouse dataset (RGB + ROMB semantic + instance + COCO panoptic) from the UnrealZoo/UnrealCV **Storagehouse** scene, for autonomous-forklift perception.

Idea: Linux + NVIDIA GPU → run a prebuilt UnrealZoo UE5.6 binary headless inside a Podman container → talk to it with the UnrealCV Python client → scripts produce labelled images. You fly through the scene from your laptop to pick good camera positions, then a script renders the full dataset.

1. Requirements

- Linux host with an **NVIDIA GPU** (≥8 GB VRAM) and driver (`nvidia-smi` works).
- **Podman** (or Docker) + NVIDIA Container Toolkit.
- ~150 GB free disk (the UE package is ~67 GB). Use a big disk, not the system drive.
- Network access to GitHub and ModelScope/HuggingFace (to download the package).
- A laptop with Python (for the interactive flythrough over SSH). Rendering happens on the GPU host; the laptop only controls it.

2. One-time setup

GPU passthrough (CDI):

```
mkdir -p ~/.config/cdi-unreal
sudo nvidia-ctl cdi generate --output=$HOME/.config/cdi-unreal/nvidia.yaml
```

Container image — create Containerfile:

```
FROM nvcr.io/nvidia/cuda:13.2.1-base-ubuntu24.04
ENV DEBIAN_FRONTEND=noninteractive NVIDIA_DRIVER_CAPABILITIES=all NVIDIA_VISIBLE_DEVICES=all
ENV XDG_RUNTIME_DIR=/tmp/xdg-runtime
RUN apt-get update && apt-get install -y --no-install-recommends \
    libvulkan1 vulkan-tools xvfb libx11-6 libxext6 libxrandr2 libxcb1 libxcb-randr0 \
    libxkbcommon0 libxi6 libxcursor1 libxinerama1 libssl2-2.0.0 libgl1 libglu1-mesa \
    libasound2t64 libpulse0 libglib2.0-0t64 ca-certificates curl unzip python3 python3-pip socat \
    && rm -rf /var/lib/apt/lists/* && mkdir -p /tmp/xdg-runtime && chmod 700 /tmp/xdg-runtime
RUN pip3 install --no-cache-dir --break-system-packages unrealcv pillow numpy
WORKDIR /workspace
```

```
podman build -t unreal-cv:latest -f Containerfile .
```

`libglib2.0-0t64` and `socat` are included on purpose — without `glib`, `import unrealcv` (which pulls in `cv2`) crashes; `socat` is needed for the interactive flythrough. If you run from a bare image instead, install both in the container before use.

Download UnrealZoo (UE5.6 Linux package, ~67 GB) from the links in <https://github.com/UnrealZoo/unrealzoo-gym> and unpack it into your work folder, e.g.:

```
/mnt/<disk>/<user>/UnrealEnv/UnrealZoo_UE5_6_Linux/Linux/UnrealZoo_UE5_6.sh
```

That `UnrealEnv` folder is mounted into the container as `/data`.

3. Run the simulator (headless)

Key facts: Vulkan needs a display → run under `xvfb-run`. UnrealCV listens on port 9104 but serves over a **unix socket** (`/tmp/unrealcv_9104.socket`), created only after the first TCP “touch”. The scene takes **5-15 min** to load. Run

the binary **interactively** (`-it`), not detached. Load the level after boot (the folder name as an argument does not work).

Terminal A — start Unreal (leave open):

```
podman rm -f uz-scene 2>/dev/null
podman run --rm -it --name uz-scene --users=keep-id \
  --cdi-spec-dir=$HOME/.config/cdi-unreal --device nvidia.com/gpu=1 \
  -v /mnt/<disk>/<user>/UnrealEnv:/data -e HOME=/tmp -p 9104:9104 \
  unreal-cv:latest \
  bash -c "xvfb-run -a /data/UnrealZoo_UE5_6_Linux/Linux/UnrealZoo_UE5_6.sh -RenderOffscreen -stdout -unattended -
ResX=1920 -ResY=1080 2>&1 | tail -80"
```

Terminal B — wait, wake the socket, load the scene:

```
until nvidia-smi | grep -qi unreal; do sleep 15; echo "...starting"; done; echo UP
podman exec uz-scene python3 -c "import socket; socket.create_connection(('127.0.0.1',9104),timeout=5).close();
print('touch ok!')"
# load the scene (if the connection drops during loading, run this line again to confirm):
podman exec uz-scene python3 -c "from unrealcv import Client; c=Client('/tmp/unrealcv_9104.socket','unix');
c.connect(); print(c.request('vget /level/name')); c.request('vset /action/game/level Storagehouse', timeout=180)"
# check:
podman exec uz-scene python3 -c "from unrealcv import Client; c=Client('/tmp/unrealcv_9104.socket','unix');
c.connect(); print(c.request('vget /unrealcv/status'))"
```

4. Generate the dataset

Copy the scripts into the `UnrealEnv` folder (= `/data`). The pipeline is four steps.

(a) Record good camera positions (laptop). Fly through the scene and press SPACE at good spots; aim for ~50-60 waypoints covering all aisles. Rendering is on the server, so use a small bridge + SSH tunnel.

```
# server (host): expose host:9105 -> container UDS, leave open
python3 uds_bridge.py
# laptop (another terminal): tunnel, leave open
ssh -N -L 9105:127.0.0.1:9105 <user>@<server>
# laptop: record (needs pip install pygame pillow numpy unrealcv)
python3 record_waypoints.py 127.0.0.1 9105 # SPACE = record, ENTER = save -> waypoints.json
```

Copy `waypoints.json` to the server's `UnrealEnv` folder.

(b) Render the dataset (server, in background).

```
podman exec -d uz-scene bash -c "cd /data && nohup python3 capture_dataset.py > cap500.log 2>&1"
tail -f /mnt/<disk>/<user>/UnrealEnv/cap500.log
```

`capture_dataset.py` varies lighting and lens angle, hides a random subset of cargo/pallets per variant, and keeps only good frames → `/data/dataset_500/`.

(c) Review and clean.

```
podman exec uz-scene python3 /data/contact_sheet.py /data/dataset_500 # overview sheets
podman exec -e DATASET=/data/dataset_500 uz-scene python3 /data/clean_void.py review 0.03
# look at void_candidates.png, then delete the ones you confirm:
podman exec -e DATASET=/data/dataset_500 uz-scene python3 /data/clean_void.py drop <id ...>
```

(d) Package into ROMB format.

```
podman exec uz-scene python3 /data/finalize_dataset.py
# -> /data/dataset_romb/1080p/ (rgb/ labels/ romb.yaml train.txt val.txt + instance/ panoptic/)
```

Download it:

```
scp -0 -r <user>@<server>:/mnt/<disk>/<user>/UnrealEnv/dataset_romb ~/Desktop/
```

5. Scripts

File	Role
<code>romb_export.py</code>	Core: build ROMB semantic + instance masks from the <code>object_mask</code> render; holds the class scheme, palette and UE-mesh → class rules.
<code>coco_panoptic.py</code>	Export COCO panoptic (id-encoded PNG + <code>panoptic.json</code>).
<code>record_waypoints.py</code>	Laptop: fly the scene and record camera positions → <code>waypoints.json</code> .
<code>capture_dataset.py</code>	Main generator: randomize lighting/FOV, hide cargo, filter quality → <code>dataset_500</code> .
<code>clean_void.py</code>	Review/remove any leftover “void” frames; keeps JSON consistent.
<code>finalize_dataset.py</code>	Repackage into the ROMB layout + train/val split + README.
<code>contact_sheet.py</code>	Tile RGBs into overview sheets for review.
<code>viz_panoptic.py</code>	Turn panoptic PNGs into readable per-instance / per-class images.
<code>uds_bridge.py</code>	Server: TCP↔UDS bridge for the laptop flythrough.
<code>flythrough_recorder.py</code>	Alternative recorder that logs a continuous camera path.

ROMB classes (0-10 + ignore=11): `drivable`, `person`, `pallet-face`, `pallet-empty`, `pallet-full`, `cargo`, `vehicle-forklift`, `vehicle-other`, `ego-forklift`, `other-object`, `vertical`. The UE→ROMB mapping is in `romb_export.py` (`CLASSIFY_RULES`); a new scene = add rules there.

6. Troubleshooting

Symptom	Fix
<code>podman ps</code> empty / only <code>xvfb-run</code>	Run interactively (<code>-it</code>), not <code>-d</code> .
<code>nvidia-smi</code> shows no unreal process	Still booting (5–15 min); or it crashed (see <code>Saved/Logs</code>).
Connection refused on 9104	Socket not created yet → TCP touch + wait for the scene.
Scene must be loaded twice	Normal: the first load drops the socket; the second confirms.
<code>import cv2 ... libgthread-2.0.so.0</code>	Install <code>libglib2.0-0t64</code> in the container.
<code>socat ...</code> not found (bridge)	Install <code>socat</code> in the container.
All images identical	You are using camera 0 → use camera 1 (FusionCam).
Mask blank / uniform	Before a mask: <code>vset /viewmode object_mask</code> , then back to <code>lit</code> for RGB.
AF_INET address must be tuple	UDS: <code>Client('/tmp/unrealcv_9104.socket', 'unix')</code> (second arg <code>'unix'</code>).
Laptop TCP client hangs on 9104	UnrealCV serves only UDS → use <code>uds_bridge.py</code> + tunnel.
Scene too dark	<code>vruntime.DefaultFeature.AutoExposure.Bias 2</code> (the generator already varies this).

Shut down: `podman rm -f uz-scene` (frees the GPU; the dataset and scripts stay on disk).